

Data Structure -

1. Linear

- a. Array
- b. Stack
- c. Queue
- d. Linked list
- e. String

2. Non linear

a. Tree

- i. Binary Tree (BT)
- ii. Binary Search Tree (BST)
- iii. Traversal of BT

- 1. Inorder
- 2. Preorder
- 3. Postorder

b. Graph

i. Graph searching techniques -

- 1. BFS
- 2. DFS

ii. Shortest path finding techniques (Dijkstra)

iii. Overcoming the vulnerabilities of Dijkstra algorithm

iv. Enhanced Shortest path finding techniques

v. Minimum spanning tree

- 1. Prim's algorithm
- 2. Kruskal algorithm

c. Hash

d. Heap

i. Binary heap creation

Algorithms -

1. Sorting
 - a. Selection
 - b. Insertion
 - c. Bubble
 - d. Quick sort
 - e. Merge sort
2. Searching
 - a. Linear
 - b. Binary
3. Greedy
4. Divide and conquer
 - a. Quick sort
 - b. Merge sort
5. Dynamic programming
 - a. Coin exchange
 - b. 0/1 Knapsack
6. Hoffman coding
7. Misc

Array -

`int A[] = {1, 2, 3, 4, 5};`

15, 1, 2, 3, 4, 10, 5, 6, 7

$O(1)$

$O(N)$

`int B[10];`

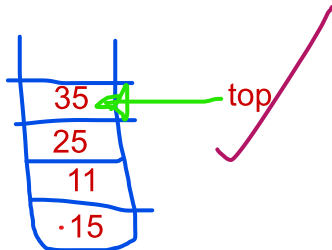
1 -> 2 -> 6 -> 3 -> 4

$O(N)$

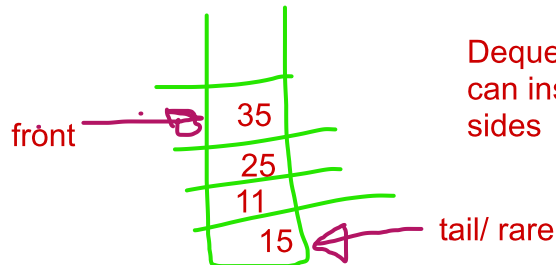
$O(N)$

`int fun(int A[], int n) // fun(B, 5)`

Stack - LIFO



Queue - FIFO



Deque -
can insert and delete from both
sides

expression = $A + B - C$

Infix = $A + B - C$

postfix (Reverse Polish) = $AB+ - C = AB+C-$

prefix (Polish) = $+AB - C = - +ABC$

$- +ABC \Rightarrow$ POSTFIX

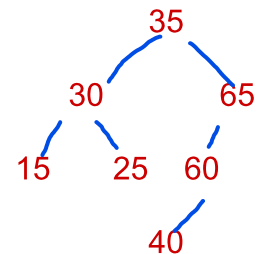
$AB+C-$

$AB-C+D/ \Rightarrow$ PREFIX

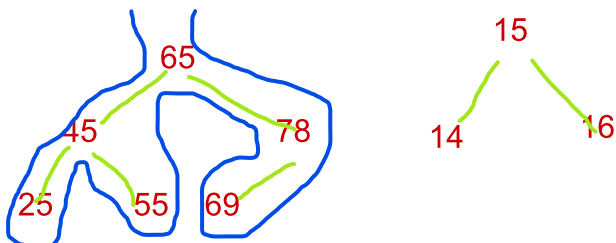
$/+ - ABCD$

Linked List -

- single (implementation, insert, delete, sort)
- double



Binary Search Tree - 65, 78, 45, 55, 69, 25



in = 15 30 25 35 40 60 65

pre order = 35 30 15 25 65 60 40

post =

in order = left -> root -> right = 25 45 55 65 69 78

post order = left -> right -> root = 25 55 45 69 78 65

pre order = root -> left -> right = 65 45 25 55 78 69